

A Cost-Benefit Study of Template Metaprogramming

1st Miguel Vezanzones Parellada

Technical University of Munich

Munich, Germany

miguel.vezanzones@tum.de

Abstract—Modern High-Performance Computing (HPC) applications rely on efficient abstractions to balance performance and usability. This work investigates how increasing compiler visibility through compile-time information affects the generated code and execution efficiency of numerical kernels. In particular, we use template metaprogramming and `constexpr` evaluation to progressively specialize implementations of the same algorithm by moving information from runtime to compile time.

The impact of compile-time specialization is evaluated by comparing tensor contraction kernels with various degrees of specialization. The evaluation considers execution time, compilation overhead, generated code characteristics, code size, and the extent to which compile-time information enables compiler optimizations such as constant propagation. The results illustrate the trade-off between abstraction, generality, and the ability of the compiler to generate efficient machine code.

Index Terms—template metaprogramming, C++, code generation, compile-time specialization, `constexpr` evaluation

I. INTRODUCTION

Modern processors are increasingly complex and utilizing them effectively requires close collaboration with the compiler. The source code must first expose sufficient information to provide the compiler with enough knowledge and visibility to generate efficient machine code. However, common high-level abstractions often hide structural properties of a program until runtime, limiting the transformations available during compilation. We refer to compiler visibility to the amount of information available to the compiler during translation that can influence optimization and code generation.

C++ aims to provide a simple and direct mapping to hardware, and zero-cost abstractions [1, p. 2] as its core philosophy. One of the main design goals of the zero-overhead principle is that the cost of using high-level abstractions should be zero, as compared to its hand-crafted counterpart [2].

Template metaprogramming allows extending the scope of this principle to reducing the cost of abstractions beyond what a programmer could produce by hand. This is achieved by guiding the compiler into generating extremely thin layers of abstraction whose complexity is resolved at compile time.

The compiler can only optimize information that is visible at compile time. Values available in `constexpr` contexts can participate in compile-time evaluation and code generation (embedding literals into assembly immediate arguments¹).

¹For example: `add eax, 1`. Here the integral literal “1” has been embedded in source, while `eax` contains a runtime value stored in a register.

Similarly, templated types allow the compiler to compute types at compile time [1, p. 6]. Template metaprogramming techniques exploit these mechanisms to compute arbitrary types and guide code generation. Template metaprogramming is often used in highly-optimized applications to afford efficient abstractions [3].

Compile-time information must be present in the source for the compiler to be able to use it. This applies most often to values, but the same thing applies to types. Compile-time information can be lowered to runtime information in a nested context, reducing the compiler’s visibility moving forward within that nested context. This process is irreversible, and defines a flow of information from its source to its destination, where it is eventually consumed.

Compile-time information can be particularly useful in specific applications such as numerical kernels, where the structure of the computation often depends on properties such as dimensions, data layout, and iteration patterns. In particular, tensor contraction has been selected as the reference application in this work because it exposes several independent dimensions of compile-time specialization. Tensor rank, tensor dimensions, and contraction indices can either be represented dynamically or embedded into the generated assembly, allowing controlled comparison between different levels of compiler visibility.

The main contribution of this work is evaluating the impact of compile-time information on runtime performance, compilation time, code size, and generated machine code. Please find the source code of this project at `constexpr_propagation`.²

II. COMPILE-TIME INFORMATION

Template metaprogramming is, fundamentally, a compile-time mechanism that allows guiding the compiler into generating specialized code in a generic way. Here, we describe the main ways in which information available to the compiler affects compile-time computation, code generation, and architectural mechanisms of computer organization.

A. Compile-Time Work

1) *Compile-Time Computation*: It is often desirable to perform computation at compile time rather than run time.

²https://github.com/MiguelVezanzones/constexpr_propagation

This applies both to values and types. Computing a value at compile time enables its use in other compile-time constructs, and the results can be directly accessed at run time. Similarly, computing types at compile time allows efficient type specialization and compile-time type-safety checks [1, p. 6].

2) *Redundant Run-Time Computation*: Sometimes, efficient API implementations expect work to be done by the caller. This often happens because work performed behind an API call must be done per call, rather than per caller. Moving such work to the compile time would solve such a problem, enabling efficient and clean APIs.

In our particular case, we allocate the output tensor and feed it to the contraction call as an output parameter for performance reasons. This is done to move a heap allocation and the output size computation logic outside of the benchmark loop.

3) *Compilation Time*: Cost of recompilation increases as the compiler is responsible for more work. Template metaprogramming techniques can become a burden if recompilation is frequent. This often happens with header-only libraries, of which multiple Boost libraries³ are classic examples.

Modern C++ standards have improved on this aspect by providing explicit language support for metaprogramming, with compile-time constructs such as C++17's `if constexpr` and C++26's `template for`. Additionally, precompiled headers and C++20 modules can reduce the cost of recompilation.

B. Code Generation

1) *Optimization*: Increasing compiler visibility extends its reasoning into possible optimizations that it can apply. For example, loop size information can enable optimizations such as automatic loop unrolling and auto-vectorization.

2) *Code Size*: Code size is affected in various ways by template metaprogramming. Most often, code grows as a result of template constructs being specialized in multiple template instantiations. However, compiled, templated code can also be simpler and smaller as complexity resolved at compile time is not preserved into the build output.

3) *Indirections*: The complexity introduced by dynamic dispatch in polymorphic data types can alternatively be resolved at compile time through compile-time polymorphism. Such a change reduces the amount of indirections needed, as function calls are statically bound instead.

4) *Type Strengthening*: The use of templates and compile-time values often allow strengthening type invariants and properties, such as replacing a `std::vector` with a `std::array`. Properties that are commonly desirable are triviality, standard layout and trivial copyability. The compiler can exploit types that meet these strict properties to generate efficient code or remove operations altogether. For example, `std::trivially-copyable`⁴ types can be efficiently

copied with `std::memcpy`, rather than through a copy assignment operator.

5) *constexpr Enabling and Propagation*: The visibility of the compiler is not reduced to the values encoded in the source code, but also to those subsequently computable. These constant, embedded values then require simpler control logic, to be accessed (accesses do not need to be thread-safe) or initialized (compile-time constants do not need to be initialized at runtime).

6) *Compile-Time Contracts*: Compile-time information allows enforcing contracts at compile time to provide strong invariant guarantees.

C. Architectural

1) *Memory Operations*: Assembly immediate arguments are not obtained from dedicated memory operations, but are rather fetched by the front-end along with their instruction. This reduces the overall number of load operations of a program. What category types and bit widths are allowed is architecture-dependent, however. Intel 64 (Intel's version of x86-64) supports small integral and no floating point-immediates, while AMD64 (AMD's version of x86-64) also supports floating-point immediates to some extent.

2) *Function Arguments*: Function arguments use architectural resources in the form of registers and stack space. Compile-time constants and static/globals, however, do not need to be provided by the caller, and are directly accessible from within the function, possibly reducing the number of run-time arguments a function needs.

3) *Memory Regions*: Compile-time information or uncertainty determines what regions of the memory model can be used. The heap is the least restrictive and can be used at run time, but with higher costs of allocation. Stack memory is fast to use, but the compiler must know the size of objects, and their lifetime is context-determined. The most restrictive section is the static/global data region, whose symbols are fixed and contained directly in the executable binary. These can be constants or read/write values. The static/global constants (located in the `.rodata` section) are the most powerful for the compiler.

Access costs for each region vary in a very minor way: heap accesses are done through general-purpose registers containing a pointer to the heap, stack accesses are done through the special-purpose stack pointer register, and static/global data accesses hard-code their target address in assembly. This progression shows how the architecture is also able to exploit available information.

4) *Register Pressure*: Immediate arguments can reduce register pressure as they never become part of the in-flight or architectural state. These are rather ephemeral values that are consumed and immediately discarded.

5) *Read-Only Data and Coherence Protocols*: Read-only data regions (`.rodata`) are simpler to manage by a multi-threaded architecture; race conditions and cache coherency do not concern read-only memory.

³<https://www.boost.org/libraries/latest/grid/>

⁴https://en.cppreference.com/cpp/types/is_trivially_copyable

III. DESIGN SPACE

This project aims to evaluate the tradeoffs of heavy template metaprogramming and `constexpr` evaluation. The design choice we focus on is: run-time versus compile-time parameters. This decision will be evaluated on its impact on performance against flexibility and compile times.

IV. COMPUTE KERNEL

The extent to which compile-time information can be exploited effectively is highly application-dependent. A reference application must be chosen such that template metaprogramming can have a direct impact on both the algorithmic structure and implementation strategy.

Tensor contraction has been selected as the reference compute kernel due to its relevance in scientific computing workloads, as well as its ability to exploit compile-time information. A tensor contraction generalizes matrix multiplication by combining two tensors over one or more shared dimensions while preserving the remaining ones as output dimensions. The operation can be expressed as a summation over a set of contraction indices:

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}$$

Unlike fixed-rank linear algebra operations, tensor contractions require handling additional structural information, including tensor rank, dimension sizes, memory layout, and contraction indices. When this information is only available at run time, the implementation must rely on dynamic mechanisms such as run-time shape traversal, and generalized iteration schemes with a multi-index. In contrast, when this information is available at compile time, the implementation can specialize the generated kernel by resolving iteration structures and intermediate values during compilation.

This makes tensor contraction a suitable benchmark application for evaluating the impact of compile-time information. For this, we consider three variants that are explained below.

V. IMPLEMENTATION

We consider three levels of compile-time information that the implementation can exploit in a meaningful way. Each level will build on top of the previous one, always elevating information from run time to compile time. These levels are referred to as *run-time rank*, *compile-time rank* and *compile-time sizes* (v1, v2, v3 respectively), according to how the available information affects each implementation.

These can be thought of as different specification standards with different requirements for compliance.

A. Run-Time Rank (v1)

This level specifies the lowest degree of compile-time information expected by each of the implementations. It requires static typing and static binding. This is a sensible baseline as it is often taken for granted.

Static typing implies that the concrete types being used are known to the compiler (*i.e.*, no `std::any` or `void*`). Static

binding implies that function calls are resolved at compile time (*i.e.*, no virtual dispatch or function pointers).

In this context, the compiler has full visibility of the concrete types and call structure, enabling optimization across function boundaries and efficient code generation. However, no algorithm-specific information such as ranks, dimensions or contraction patterns will be accessible during code generation.

1) *v1::tensor(t1)*: This is a tensor implementation that adheres to the v1 specification. This variant must use the heap to store both its data and its metadata since its rank is known only at run time. The data buffer will be a contiguous, random-access buffer. Similarly, the metadata buffer will contain the tensor sizes and strides for efficient access with a multi-index. The memory layout of this implementation is suboptimal in terms of locality, as there will be 7 different allocations⁵ which will lead to inefficient utilization of memory subsystem resources.

2) *v1::contraction(c1)*: Since tensor ranks are unknown at compile time, the first-level contraction implementation uses a multi-index to iterate over them. The multi-index uses an internal `for` loop for each increment, introducing significant overhead and uncertainty in iteration.

B. Compile-Time Rank (v2)

The second level requires that the rank of the tensors and the contraction degree be known at compile time.

1) *v2::tensor(t2)*: Knowing the rank of the tensor allows the strides and sizes involved to be stored within the tensor object itself. Thanks to this compile-time information, the tensor metadata will be stored in the stack, rather than the heap, and accessible without a pointer indirection.⁶ The data itself must still be heap-allocated, however, as the size of each dimension is not known.

2) *v2::contraction(c2)*: Knowing the rank of the tensors significantly simplifies iteration.⁷ This implementation sets up two sets of nested `for` loops using template metaprogramming, eliminating the need for a multi-index.

The internal structure of this iteration scheme is shown in Listing 1. Given a range of sizes of size `Rank`, this function stamps one nested loop for each dimension and keeps track of the indices per dimension. In the inner most loop, a callback is invoked, adding the iteration indices as an additional argument.

C. Compile-Time Sizes (v3)

The third level requires that the sizes of the tensors and the indices to contract over be known at compile time.

⁵Two allocations for each tensor and one for the contraction index set. Each tensor uses two buffers in the heap, these could be merged into one, but the additional complexity was deemed unimportant.

⁶Moving data from the stack to the heap also removes a costly memory allocation, but these are not included in the benchmark loop.

⁷The rank of a tensor corresponds to the number of nested `for` loops required to iterate over it in a structured way.

```

template <
    std::integral Size_Type,
    Size_Type I,
    Size_Type Rank>
constexpr auto shaped_for_impl(
    std::ranges::sized_range auto const& sizes,
    std::ranges::sized_range auto& idxs,
    auto& fn,
    auto&&... args
) noexcept -> void
{
    if constexpr (I == Rank)
    {
        std::invoke(
            std::forward<decltype(fn)>(fn),
            std::forward<decltype(args)>(args)... ,
            idxs
        );
        return;
    }

    for (idxs[I] = 0; idxs[I] != sizes[I];
         ++idxs[I])
    {
        shaped_for_impl<Size_Type, I + 1, Rank>(
            sizes,
            idxs,
            std::forward<decltype(fn)>(fn),
            std::forward<decltype(args)>(args)...
        );
    }
}

```

Listing 1. Nested loop iteration.

1) $v3::tensor(t3)$: This affects the tensor implementation in two major ways. First, the data in the tensor may be stored in the stack, significantly improving locality for small tensors. Additionally, tensor sizes and strides become compile-time constants that are part of the tensor type itself, reducing the size of each instance.

A limit of 128 KB per tensor has been used to conditionally store the data in the heap to prevent stack overflow.

2) $v3::contraction(c3)$: Knowing the size of the tensors allows propagating these values into the loop limits of the two sets of nested `for` loops. This enables inlining the loop limits into the assembly as integral immediate arguments, and enables aggressive loop unrolling and vectorization.

Additionally, iteration functions now require less input arguments, since the location of static data is known at compile time. This change makes feasible computing the target linear index for each tensor during iteration, removing an inner product computation between the strides and the multi-index on each tensor access.⁸

D. Signatures

Listing 2 shows the signatures of the tensor types and the tensor contraction.⁹

To allow generic use of the contraction implementations, template arguments are deduced from the input arguments.

⁸This same optimization was unsuccessful for the $v2::contraction$ iteration scheme, presumably for the significant increase in function arguments this requires.

⁹Names have been simplified.

```

using T = double; // value_type

std::size_t N = \* *\; // Rank
std::size_t Sx = \* *\; // N Sizes

// Tensor signatures
using t1 = v1::tensor<T>;
using t2 = v2::tensor<T, N>;
using t3 = v3::tensor<T, S1, ..., SN>;

// Tensor contraction signatures
v1::contract(v1::compatible auto const& A,
             v1::compatible auto const& B,
             v1::compatible auto & C,
             v1::index_set auto const& cis);

v2::contract(v2::compatible auto const& A,
             v2::compatible auto const& B,
             v2::compatible auto & C,
             v2::index_set auto const& cis);

v3::contract<CIS>(v3::compatible auto const& A,
                 v3::compatible auto const& B,
                 v3::compatible auto & C);

```

Listing 2. Tensor and contraction simplified signatures.

Only $v3::contraction$ requires an additional Non-Type Template Parameter (NTTP) argument as it is necessary to determine the shape of the output tensor, which must be known at compile time.

E. Compatibility

Contraction and tensor implementations of different levels are compatible because each level extends the previous one with stricter requirements, and compile-time information can always be lowered to run-time information. Since the opposite is not true, compatibility goes in one direction, making $v3::tensor$ compatible with any contraction implementation, $v2::tensor$ compatible with $v2::contraction$ and $v1::contraction$, and $v1::tensor$ only compatible with $v1::contraction$.

This will be used to assess what part of the performance improvements can be attributed to changes in the tensor implementation or the contraction implementation.

VI. EXPERIMENTAL SETUP

All benchmarks were conducted on a system with the specifications shown in Tab. I. Benchmarks were executed sequentially and pinned to the same, single core in all cases. Parallel execution of the benchmarks would be fine for small cases, but for sizes over the L2 cache size, these would interfere with one another through the L3.

The compiler was configured with the following aggressive optimization settings to focus benchmark differences on code generation:

```

-O3 -fno-exceptions -march=native -mavx
-fno-math-errno -ffast-math -fassociative-math
-fno-trapping-math -fltto

```

TABLE I
HARDWARE AND SOFTWARE ENVIRONMENT USED FOR BENCHMARKING.

Operating System	Arch Linux
Kernel	7.1.2-arch3-1
Compiler	GCC 16.1.1
Language Standard	C++26
CPU	Intel Core i7-9750H
L1D / L2 / L3 Cache	32KB / 256KB / 12MB
DRAM	16 GB DDR4-2667 MT/s
Base Frequency	2.6 GHz

These flags enable high-level, whole-program optimization, target the native CPU architecture, enable autovectorization, allow aggressive floating-point transformations and inlining, and reduce error-handling code. Note, however, that `-ffast-math` enables possibly unsafe optimizations that are not compliant with the IEEE 754 Standard for Floating-Point Arithmetic.¹⁰

VII. RESULTS

Elevating information from run time to compile time leads to a series of trade-offs, and must be evaluated with metrics that capture them. We will primarily use the following set of objective measurements

- `runtime`: Main metric to measure code efficiency.
- `compilation time`: Measure how compilation times increase with compile-time computation and template metaprogramming.
- `code size`: Measure the impact of template metaprogramming in binary size.

Additionally, we will look at the generated assembly to observe the effects of constant propagation and the presence of immediate arguments.

A. Runtime

Most often, compile-time information is exploited to improve performance at the expense of flexibility. Runtime is, therefore, the most important metric we will analyze in order to measure potential gains. Whether those gains are justifiable depends on the application and should be carefully considered.

The benchmark consists of 63 contractions of tensors of various sizes and ranks. Sizes range from 2 to 256 per dimension, ranks range from 2 to 16, and contracted ranks range accordingly. Not all combinations are present in the benchmark as exponential growth rapidly exhausts system resources.

Runtime is plotted against the number of floating point operations needed for the naive implementation we are using, which amounts to two floating point operations per inner-most loop iteration.

For the following plots, C1, C2, C3 stand for the three contraction implementations, and T1, T2, T3 stand for the three tensor implementations.

Runtime has been measured using Google Benchmark with a minimum runtime of 2 seconds per data point.

¹⁰<https://ieeexplore.ieee.org/document/8766229>

1) `v1::contraction`: Fig. 1 shows how the tensor implementation impacts performance for the same contraction algorithm that does not exploit the specific tensor implementation. The difference lies in the overhead of accessing tensor data and metadata.

The improvement obtained by replacing `v1::tensor` with `v2::tensor` is relatively small. The main effect responsible for the improvement should be improved locality for the tensor metadata, which is now stored in the stack along the metadata of the other tensors.

Using `v3::tensor` as the tensor implementation leads to more significant improvements. Now, tensor indexing should be much more efficient, as the strides are compile time constants that the compiler can inline. Additionally, tensor data is stored in the stack for small tensors, which improves performance significantly for sufficiently small tensors.

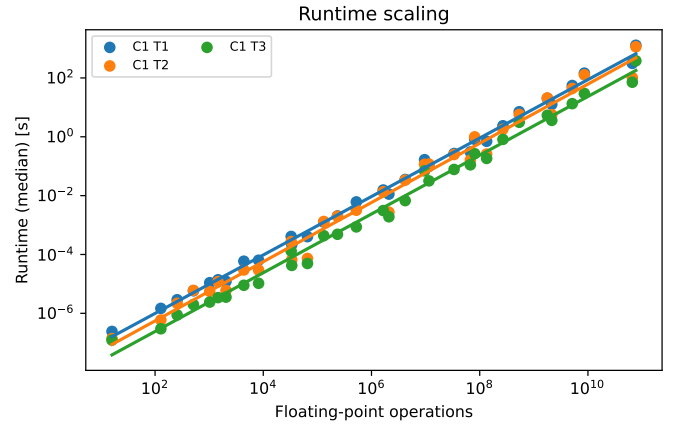


Fig. 1. `v1::contraction` performance scaling with each compatible tensor.

2) `v2::contraction`: This implementation can use the rank of the tensors and contraction index set at compile time, bypassing the need to iterate with multi-indices and removing allocations entirely from the benchmark loop. Here, the use of `v3::tensor` does not seem to improve upon the pure `v2` version even though each tensor access should be more efficient. Fig. 2 shows the results graphically.

This unexpected result indicates that the improvements in locality from moving small tensors to contiguous memory in the stack, and the possibility to inline tensor strides in each tensor access did not lead to meaningful performance improvements. In both cases, the memory accesses that can be avoided, or whose locality can be improved correspond to highly-accessed symbols that should be fast to access most of the time.

3) `v3::contraction`: This contraction implementation is only compatible with `v3::tensor`, as it expects compile time sizes from the input and output tensors. For completeness, Fig. 3 shows the scaling of this implementation graphically. This implementation propagates constants all the way to the loop control limits.

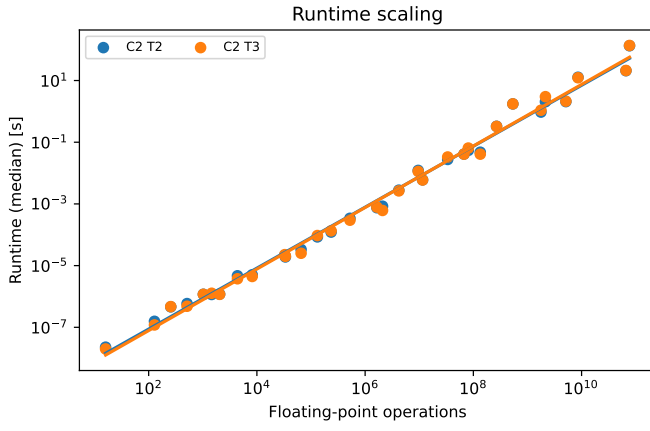


Fig. 2. `v2::contraction` performance scaling with each compatible tensor.

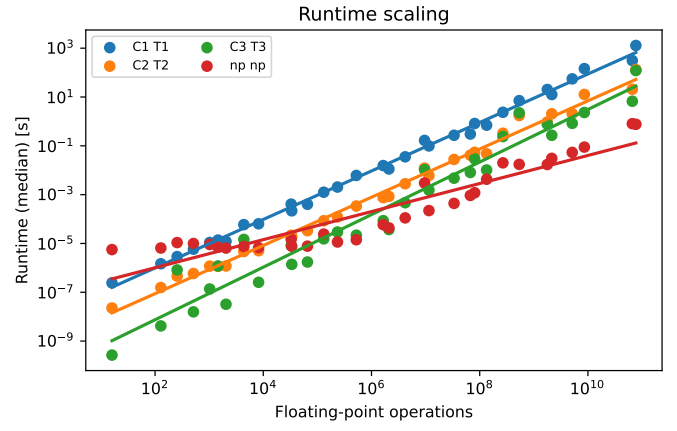


Fig. 4. Contraction performance for each of the compile-time information variants.

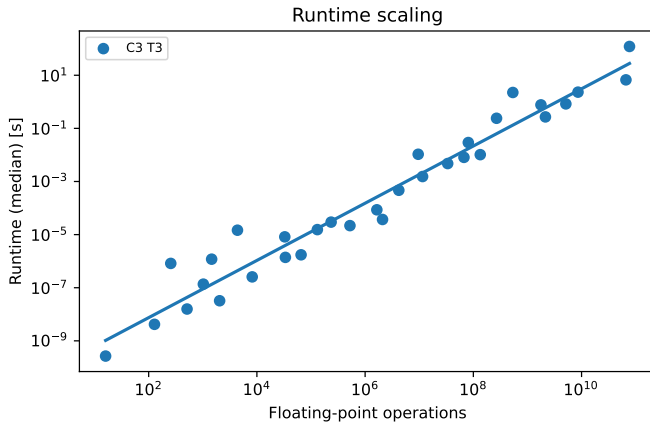


Fig. 3. `v3::contraction` performance scaling.

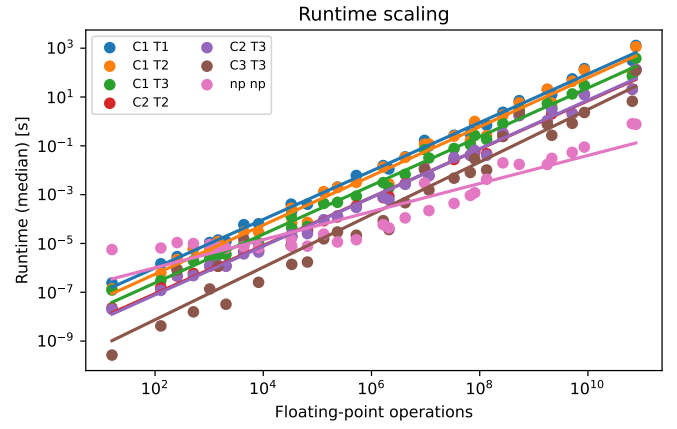


Fig. 5. Contraction performance for all compatible tensor and contraction implementations.

4) *Overview:* Fig. 4 shows the performance of each of the three variants, along with NumPy’s `tensor_dot` for reference (denoted by “NumPy”).

A direct comparison with NumPy is not fair, as its implementation is fully generic and most likely respects IEEE 754 floating point arithmetic. However, it serves as an interesting comparison with an optimized library, to show the value of tight implementations in HPC projects.

NumPy has a significant overhead from function calls, preprocessing to optimize iteration patterns (such as permuting the order of loops), which is only amortized for small to medium size tensors.

Finally, Fig. 5 gathers all plots for completeness.

tab. II shows the geometric mean of the median runtime across the benchmark data points obtained with the two compilers tested. This data is shown for reference, as an analysis on code generation for each of the cases is outside the scope of this work. The similarity of the runtime results, however, suggest that both compilers are capable of performing similar optimizations.

B. Compilation Time

Template metaprogramming demands the compiler to perform significant type manipulations, and `constexpr` evaluation requires the compiler to execute unoptimized C++ code in an internal interpreter. As a result, we would expect compile times to increase as tensor rank increases for the more template-heavy versions. Compile times for the benchmark examples are shown in Fig. 6.

Compilation time results show that `v3::tensor` introduces a significant overhead, increasing compile times by up to four times.

This increase in compile time was expected and not particularly problematic for these small micro benchmarks, but could become a problem in larger, more sophisticated applications. The real limitation, however, is that compile time computation cannot scale to the same order of magnitude as run-time computation can. Compile-time computation is not a replacement, but a complement to run-time computation. Here, template metaprogramming is used to define a program specification that will allow its compiled counterpart to be as efficient as

TABLE II
GEOMETRIC MEAN OF THE MEDIAN EXECUTION TIME (S).

Contraction	Tensor	GCC 16.1.1	Clang 22.1.6
C1	T1	6.06×10^{-3}	5.98×10^{-3}
C1	T2	3.25×10^{-3}	1.94×10^{-3}
C1	T3	1.47×10^{-3}	2.97×10^{-3}
C2	T2	5.11×10^{-4}	9.47×10^{-4}
C2	T3	5.00×10^{-4}	6.45×10^{-4}
C3	T3	8.00×10^{-5}	2.57×10^{-4}
NumPy	—	—	1.63×10^{-4}

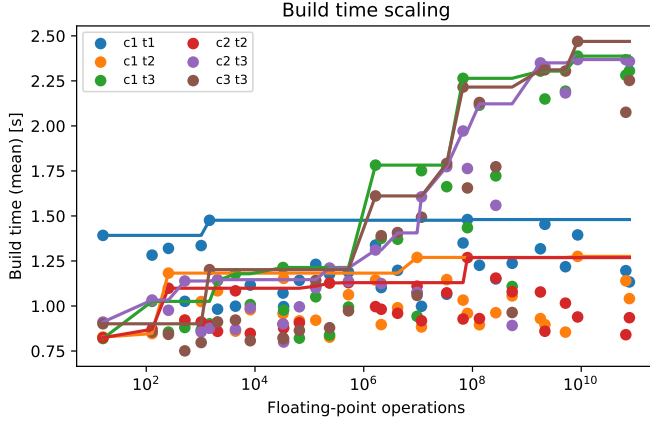


Fig. 6. Build time of each of the benchmark examples.

possible in executing real workload.

C. Code Generation

In terms of code generation, we will analyze constant propagation into immediate arguments in the v3 variant, auto-vectorization and code size.

1) *Assembly immediates*: We only consider this variant because it is the only implementation where the compiler has enough information such that it should not embed it everywhere. This point will be illustrated below.

For v2, all of the compile-time information provided is explicitly exploited, and is, therefore, not up to the implementation to embed in the assembly. An example of this scenario are the nested `for` loops shown in Listing 1; the implementation is not allowed to choose an equivalent alternative that does not contain them, as we have explicitly stamped nested loops. Similarly, the stack arrays that contain tensor metadata are explicitly instantiated, so the compiler is forced to use the compile time rank specified by the v2 variant. The compiler is free to optimize the code in multiple ways that would essentially remove these constructs we have explicitly requested, but in such cases the compiler is choosing to transform code that already has these values embedded.

In v3, however, we, for example, explicitly propagate `constexpr` values into the limits of `for` loops. If these loop limits were embedded in the assembly, there would need to be an assembly block for each such loop, as each must contain

their own loop limits. However, since we are using regular, runtime loops¹¹, we are not forcing the compiler into stamping many variations of the same loop, rather, we are providing enough information for it to be possible. The compiler will then decide whether to inline `constexpr` values (such as the loop limits) in the assembly based on its trade-offs, primarily runtime performance and code size.

In the generated assembly of v3 examples, we have observed that loop limits and tensor strides have been embedded in the assembly as immediate arguments in most cases. Loop limits are simple to embed, as the compiler can actively exploit a regularity present in most of the benchmark data points. In particular, we mostly used tensors with the same size per dimension, which significantly reduces the times that code duplication would happen by embedding values in assembly (*i.e.*, if the same value is always embedded, then no duplication is necessary). However, even these regular tensors have different strides per dimension, which makes it interesting to observe that the compiler decided to inline them too, leading to code duplication, rather than loading these values from memory and reusing basic blocks.

Annotated assembly snippets are shown in Listings 3 and 4.

```
# for (idxs[I] = loop_t::start(I); idxs[I] != loop_t::
  :end(I);
56:    cmp    $0x10, %rbx

# out_idx += C_Strides[I];
45:    add    $0x100, %rbp

# a_base += A_Strides[I];
48:    add    $0x10, %rbx
```

Listing 3. Example with (16^3) tensors: Assembly-inlined loop limits and tensor strides

```
for (idxs[I] = loop_t::start(I); idxs[I] != loop_t::
  end(I);
56:    cmp    $0xf, %rbx

# out_idx += C_Strides[I];
3f:    add    $0xc5c1, %r12

# a_base += A_Strides[I];
46:    add    $0xf, %rbp
```

Listing 4. Example with (15^6) tensors: Assembly-inlined loop limits and tensor strides

2) *Auto-Vectorization*: A simple auto-vectorization analysis of the source showed that all builds had generated SIMD instructions, with far more instances in the v1 and v2 variants than in v3. This interesting observation matches with build sizes discussed below. Additionally, the number of SIMD instructions in v2 and v3 increased with the rank of the tensors as expected.

3) *Build Size*: Build sizes for each of the benchmark sample points and variants are shown in fig. 7.

Results show that build size decreases as more compile-time information is available. In some cases this should correspond to removed dependencies (such as `std::vector`). Part of the significant gap between v2::contraction and

¹¹As opposed to the compile time recursion loop used in Listing 1 to stamp `for` loops.

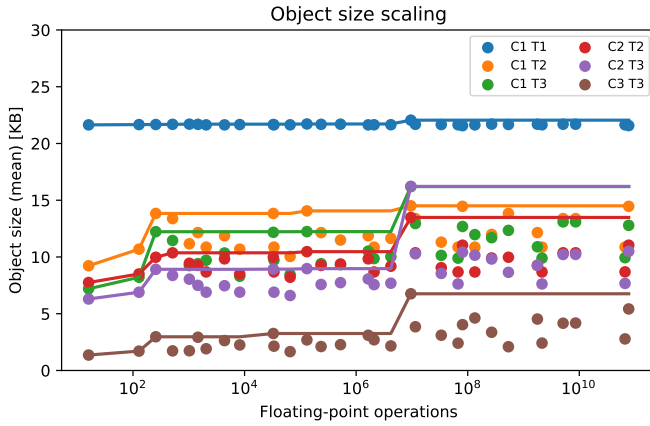


Fig. 7. Build sizes of each of the benchmark examples.

`v3::contraction` can be explained by the fact that `v3` does not use `std::inner_product` to compute the linear index in each tensor access.¹²

Further assembly analysis shows that `v3` assembly is highly optimized, especially for small tensor sizes, to the point of being mostly vector instructions and no function calls or loops. The implications of such differences in code size are hard to assess, as they will depend on how this memory is used. However, it is safe to assume that the smaller-sized builds will enjoy a better spatial and temporal locality both in the front-end and back-end processor fetch units.

VIII. DISCUSSION

The different amounts of compile-time information embedded in source led to consistent differences in runtime performance ranging from one to three orders of magnitude across the benchmark suite. These improvements are attributed to how compile time constants contribute to thinner layers of abstraction, influence optimization passes, and allow fewer resources being allocated to function calls, control flow logic and runtime setup routines.

However, these operations become memory bound as tensors grow, which gradually reduces the importance of small fixed costs. Memory boundness is determined by cache sizes, but also access patterns, blocking strategies and such constructs which we do not explicitly optimize for. Consequently, the runtime performance of all variants tends to converge as tensor sizes grow.

A high-performance tensor contraction implementation should consider cache behaviour and vectorization above constant propagation, while keeping their relationship in mind. Constant propagation can reduce the overall number of memory operations needed for a program, primarily through assembly literals and by promoting stack and static global memory use. Similarly, compiler visibility may enable auto-vectorization, which can be critical for performance.

¹²`v3::contraction` computes the linear index on the fly during iteration.

IX. CONCLUSION

This study into compile-time information and runtime performance of numerical kernels leads to an obvious question: Under what circumstances should high-performance applications embed constants at compile time? An answer to this question is highly dependent on the kind of software in question.

High-performance numeric packages intended for distribution as binaries, such as a BLAS implementation, will not have access to application-specific constants and have, therefore, no choice.

As for in-house software and header libraries, the performance difference is only meaningful if the application is intended to run for longer than it will compile. Simulation software is often well suited to compile-time specialization because long execution times can amortize increased compilation costs. In view of the presented results, simulation software should often consider embedding constants in their hot, numeric kernels as potential performance gains more than offset any increase in compilation costs for long-running executions. Note that this does not sacrifice the flexibility of the simulation package in a meaningful way, as these constants are exploited by the compiler through template-metaprogramming techniques in a generic way for the developer. In this case, a significant concern is the potential increase in the cost of the iteration loop, especially important during development. A sensible implementation could be flexible enough to choose the point in which compile time information is lowered to the runtime, which then enables the possibility to use the most expensive compile-time constructs at compile time.

This paper has highlighted the importance of template metaprogramming in HPC applications as a tool to drive the compiler into generating efficient machine code and thin abstractions. In any case, these are powerful tools to be aware of and use with care.

REFERENCES

- [1] B. Stroustrup, *Foundations of C++*, Berlin Heidelberg, 2011. [Online]. Available: <https://www.stroustrup.com/ETAPS12-corrected.pdf>
- [2] *Zero-overhead principle* - cppreference.com. Accessed: Jun. 26, 2026. [Online]. Available: https://en.cppreference.com/cpp/language/Zero-overhead_principle
- [3] A. Hendricks, “Lessons Learned: Template-Heavy C++ in Production HPC Runtime Systems,” en, in *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, St Louis MO USA: ACM, Nov. 2025, pp. 1009–1016, ISBN: 979-8-4007-1871-7. DOI: 10.1145/3731599.3767476 Accessed: May 6, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/3731599.3767476>